
PORTFOLIO - GRADUAAT PROGRAMMEREN

Persoonlijk Dashboard

Een webapplicatie met vier API-gedreven widgets

De beheerder · Doel 4: Je lost softwareproblemen op (troubleshooting)

Auteur Jonas Trap

Opleiding Graduaat Programmeren, Thomas More Geel

Academiejaar 2025-2026

Type bewijs eigen project

Het project

Dit dashboard is een persoonlijk project: een single-page webapplicatie die vier dagelijkse noden samenbrengt op één centrale pagina. De vier widgets tonen het actuele weerbericht voor opgeslagen locaties, een dagelijks avondmaalrecept met filters voor dieet en bereidingstijd, de NASA-foto van de dag met uitleg, en een eenvoudig takenbeheer.

Het project dient als demonstratiestuk van mijn vaardigheden in webontwikkeling en API-integratie. De nadruk lag bewust op een nette, professionele uitwerking die de geleerde concepten concreet toepast: een duidelijke architectuur, type-veiligheid, integratie met externe API's en degelijke foutafhandeling.

Technische keuzes

- **Build tool:** Vite, voor een snelle ontwikkelomgeving met native TypeScript-ondersteuning.
- **Taal:** TypeScript, voor type-veiligheid en betere onderhoudbaarheid.
- **CSS-framework:** Bootstrap 5, voor een degelijke grid en ingebouwde dark mode.
- **API's:** OpenWeatherMap (weer), Spoonacular (recepten), NASA APOD (foto van de dag), en een vertaal-API voor de Nederlandse inhoud.

Broncode

De volledige broncode van dit project staat publiek op GitHub, zodat ze altijd terug te vinden is: github.com/jonastapmore/jonastap-projecten/tree/main/dashboard

Gestructureerd debuggen

Ik koppel dit project aan dit doel omdat ik tijdens de ontwikkeling meerdere echte problemen ben tegengekomen en die telkens op dezelfde gestructureerde manier heb aangepakt: eerst het **gedrag en de gegevens analyseren**, dan de **oorzaak aantonen**, vervolgens een **oplossing implementeren** en ten slotte de **aanpassing documenteren**. Hieronder werk ik drie concrete cases uit volgens die vier stappen.

Case 1: NASA-foto laadt soms niet (503-fouten)

Gedrag en gegevens. De NASA-widget toonde af en toe een foutmelding, terwijl een herlaad even later wel werkte. Bij het testen zag ik in de netwerkrespons wisselende statuscodes: vaak 503, en dan plots weer 200.

Oorzaak. Het lag niet aan mijn code maar aan de NASA APOD-API zelf: die is wisselvallig en geeft tijdelijk een serverfout (5xx) terug. Een eenmalige aanvraag faalt dan onterecht.

Oplossing. Ik bouwde een automatische herhaling in: bij een 5xx-status wacht de widget kort en probeert hij opnieuw, tot maximaal vier pogingen. Pas daarna toont hij een nette foutmelding.

Documentatie. De retry kreeg commentaar in de code en de werking werd opgenomen in de README. Het commit-bericht van de NASA-widget vermeldt de retry uitdrukkelijk, zodat de aanpassing terugvindbaar is in de git-historie.

Case 2: Witte pagina na het online plaatsen (MIME-fout)

Gedrag en gegevens. Lokaal werkte alles, maar online (jonastrap.be/dashboard) bleef de pagina wit. De browserconsole gaf een MIME-fout: de JavaScript-modules werden gereserveerd als text/html in plaats van als script.

Oorzaak. De app draait in de submap /dashboard/, maar de build verwees met absolute paden naar de domeinroot. Daardoor vond de browser de bestanden niet en kreeg hij de 404-foutpagina (HTML) terug in plaats van het script.

Oplossing. Ik stelde het basis-pad in de Vite-configuratie in op /dashboard/, zodat alle asset-paden naar de juiste submap wijzen. Daarna opnieuw gebouwd en geüpload.

Documentatie. De reden staat als commentaar bij de instelling in vite.config.ts, zodat de keuze later meteen duidelijk is.

Case 3: Receptwidget verdwijnt bij het laden

Gedrag en gegevens. Na het hernoemen van een filter was de volledige receptwidget weg bij het laden van de pagina, met een foutmelding in de console.

Oorzaak. Bij het hernoemen waren er nog verwijzingen naar de oude private field blijven staan. Een onbestaande private field veroorzaakt een runtime-fout die het hele component laat crashen.

Oplossing. Ik spoorde alle oude verwijzingen op en verving ze consequent door de nieuwe naam, waarna de widget weer correct laadde.

Documentatie. De correctie werd tijdens de ontwikkeling doorgevoerd en zit mee in de commit waarmee de receptwidget werd afgewerkt.

Het resultaat

De oplossingen zijn niet alleen bedacht maar ook geïmplementeerd en gedocumenteerd. Hieronder de code van de twee belangrijkste cases.

Case 1: automatische herhaling bij een tijdelijke storing

```
async #getPicture(poging = 1): Promise<void> {
  // ...laadscherm tonen...
  const res = await fetch(
    `https://api.nasa.gov/planetary/apod?api_key=${apiKey}&thumbs=true`,
  );

  // Tijdelijke serverfout (5xx) van NASA: even wachten en
  // opnieuw proberen (max 4 pogingen)
  if (res.status >= 500 && poging < 4) {
    await new Promise((r) => setTimeout(r, 1000));
    return this.#getPicture(poging + 1);
  }

  if (!res.ok) {
    this.#showError('Kon de NASA-foto van vandaag niet laden.');
```

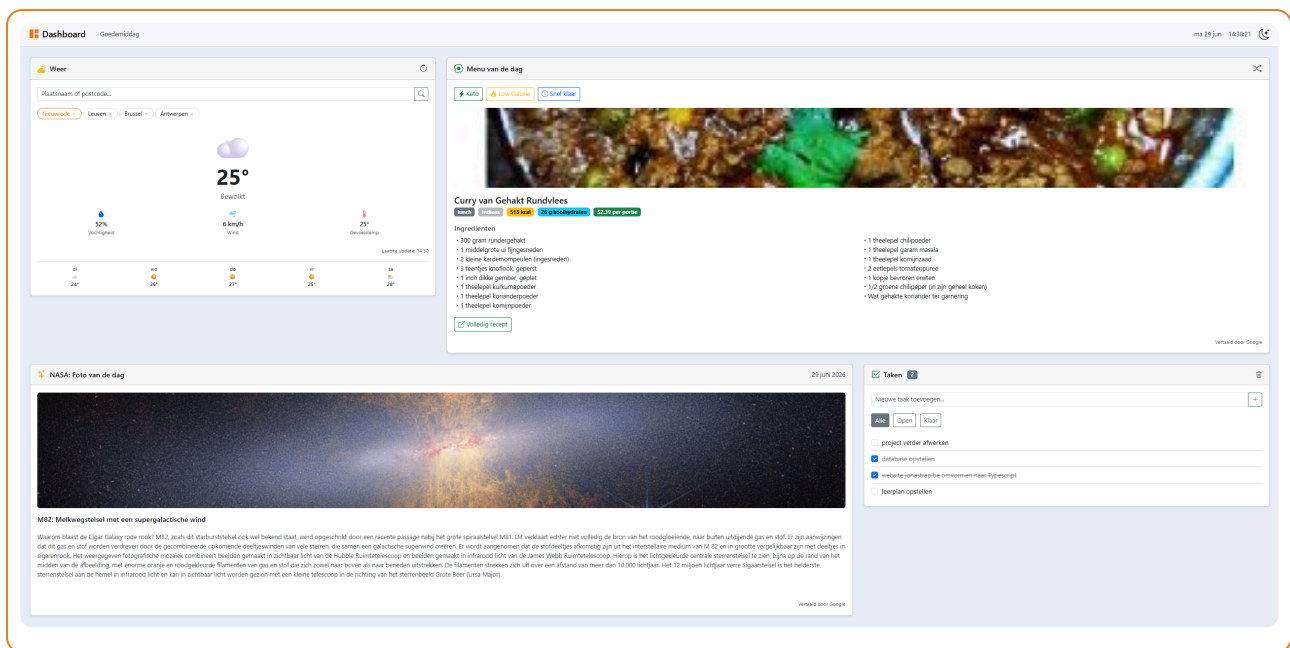
Fragment uit src/components/nasaWidget/nasaWidget.ts.

Case 2: het juiste basis-pad voor de submap

```
import { defineConfig } from 'vite';

export default defineConfig({
  // De app draait online in de submap /dashboard/, dus moeten de
  // asset-paden daarnaar verwijzen (anders zoekt de browser ze op
  // de domeinroot).
  base: '/dashboard/',
});
```

Het volledige bestand vite.config.ts, met commentaar dat de oplossing verantwoordt.



Afbeelding 1: het dashboard werkt online correct nadat de problemen waren opgelost.

Zelfreflectie

Wat ging goed

Ik loste de problemen niet op met gokwerk, maar door eerst het gedrag te observeren en de gegevens (statuscodes, console-meldingen) te bekijken. Daardoor vond ik telkens de echte oorzaak in plaats van enkel het symptoom te bestrijden.

Wat ik geleerd heb

Ik leerde het verschil tussen een fout in mijn eigen code (de hernoemde private field) en een fout buiten mijn code (de wisselvallige NASA-API, de hosting-omgeving). De aanpak verschilt: het ene los je op door de code te corrigeren, het andere door veerkracht in te bouwen (een retry) of de configuratie af te stemmen op de omgeving.

Uitdagingen

De MIME-fout was lastig omdat alles lokaal perfect werkte: het probleem dook pas op in de online omgeving. Dat dwong me om verder te kijken dan mijn eigen machine en de volledige keten (build, hosting, paden) te onderzoeken.

Wat beter kan

- De productie-build vroeger en vaker testen, zodat omgevingsgebonden fouten zoals de MIME-fout eerder aan het licht komen.
- Een logboek van opgeloste problemen bijhouden, zodat terugkerende fouten sneller herkend worden.

Conclusie

Dit project toont aan dat ik **gestructureerd debug**: ik analyseer gedrag en gegevens, toon de oorzaak aan, implementeer een oplossing en documenteer de aanpassing, met commentaar in de code, de README en duidelijke git-commits.